`Assignment 3` Optimization Problems

Shuifan Wang `shepherd.wang@foxmail.com`

April 10, 2025

Problem 1

The Minimum of Schaffer Function.

$$f(x,y) = 0.5 + \frac{\sin^2\sqrt{x^2+y^2} - 0.5}{(1+0.001(x^2+y^2))^2}$$

The result obtained using the genetic algorithm is not very satisfactory, with the minimum value being trapped in a local convergence at $(x,y) = (-2.3724, -2.0547)$, $f(x,y) = 0.009716$. However, using the multi-start `fminunc` directly yielded the correct result $(x,y) = (0,0), \quad f(x,y) = 0$, as the starting points happened to include the location of the global minimum.

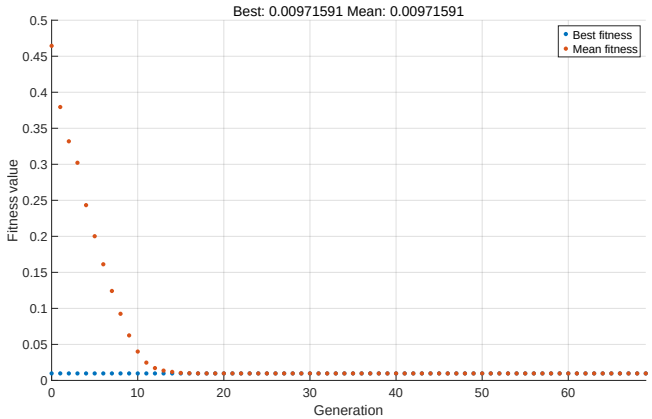

Figure 1: Problem 1

```
%----------Problem 1----------
diary('output1.txt');
diary on;

clc; clear; close all;

% First-Order Optimality Conditions for Trust-Region Algorithm

x0_list = [10, 10; -10, -10; 0, 0; -10, 10; 10, -10];
schaffer = @(x) 0.5 + ((sin(sqrt(x(1)^2 + x(2)^2)))^2 - 0.5)/(1 + ...
    0.001*(x(1)^2 + x(2)^2))^2;
options = optimset;
options.TolFun = 1.0e-10;
options.Tolx = 1.0e-10;
options = optimset('PlotFcns', @optimplotfirstorderopt);

best_FVAL = Inf;
best_X = [];
for i = 1:size(x0_list, 1)
    x0 = x0_list(i, :);
    [X, FVAL] = fminunc(schaffer, x0, options);
    if FVAL < best_FVAL
        best_FVAL = FVAL;
        best_X = X;
    end
end
fprintf('Optimal Solution: x = %.4f, y = %.4f, f(x,y) = %.6f\n', ...
    best_X(1), best_X(2), best_FVAL);

clc; clear; close all;

% Genetic Algorithm

hybrid_options = optimoptions('fmincon', 'Algorithm', 'sqp');
options = optimoptions('ga', ...
    'MaxGenerations', 5000, ...
    'PopulationSize', 1000, ...
    'CrossoverFraction', 0.8, ...
    'MutationFcn', @mutationadaptfeasible, ...
    'FunctionTolerance', 1e-12, ...
    'HybridFcn', {@fmincon, hybrid_options}, ...
    'PlotFcn', @gaplotbestf);
lb = [-10, -10];
ub = [10, 10];
schaffer = @(x) 0.5 + ((sin(sqrt(x(1)^2 + x(2)^2)))^2 - 0.5)/(1 + ...
    0.001*(x(1)^2 + x(2)^2))^2;

[X, FVAL] = ga(schaffer, 2, [], [], [], [], lb, ub, [], options);

fprintf('Optimal Solution: x = %.4f, y = %.4f, f(x,y) = %.6f\n', ...
    X(1), X(2), FVAL);
set(gca, 'FontSize', 18);
set(gcf, 'Position', [100, 100, 1600, 900]);
exportgraphics(gcf, 'Problem1_ga.pdf', 'ContentType', 'vector');

diary off;
```

Problem 2

$$\min f(x) = 10x_1^3 + x_1x_2^2 + x_3(x_1^2 + x_2^2)$$
$$\text{s.t.} \begin{cases} \sqrt{x_1^2 + x_2^2} - x_3 - 10 \leq 0 \\ \sqrt{x_1^2 + x_2^2} + x_3 - 3 \leq 0 \end{cases}$$

The results obtained using `fmincon` or simulated annealing algorithm (with penalty function method) is consistent with the result calculated theoretically: $(x_1, x_2, x_3) = (-6.5, 0, -3.5), f = -2894.124996$. While the result of pattern search algorithm is $(x_1, x_2, x_3) = (-3.3166, -5.0000, -4.0000), f = -591.745018$, far from satisfactory. It also indicates that the start point might influence. Moreover, simulated annealing converges with a slow trend after 1000 times of iteration. Therefore, it is advisable to first use simulated annealing algorithm to determine the start point and then use pattern search algorithm.

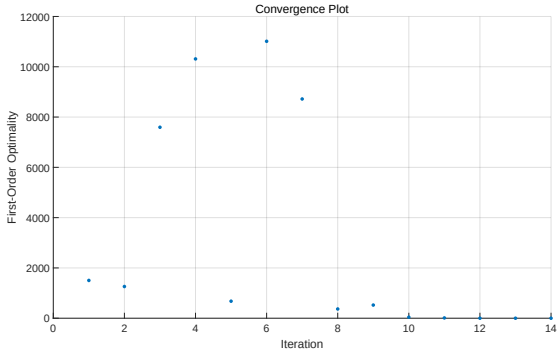

Figure 2: Problem 2tr

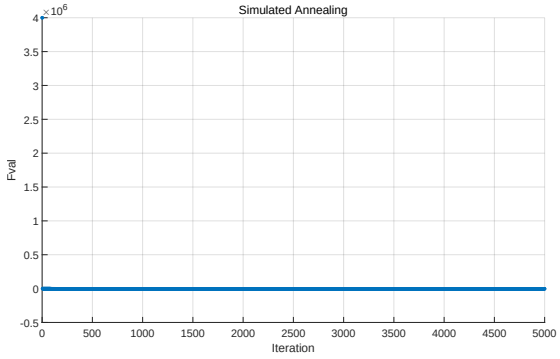

Figure 3: Problem 2sa

```
%----------Problem 2----------
diary('output2.txt');
diary on;

clc; clear; close all;

% First-Order Optimality Conditions for Trust-Region Algorithm

f = @(x) 10*x(1)^3 + x(1)*x(2)^2 + x(3)*(x(1)^2 + x(2)^2);
nonlcon = @(x) deal([ sqrt(x(1)^2 + x(2)^2) - x(3) - 10;
                      sqrt(x(1)^2 + x(2)^2) + x(3) - 3 ],  []);
x0 = [0, -5, 0];
options = optimset;
options.TolFun = 1.0e-10;
options.Tolx = 1.0e-10;
options = optimset('PlotFcns', @optimplotfirstorderopt);

[X, FVAL, EXITFLAG, OUTPUT] = fmincon(f, x0, [], [], [], [], [], ...
    [], nonlcon, options);

fprintf('Optimal Solution: x1 = %.4f, x2 = %.4f, x3 = %.4f, f = ...
    %.6f\n', X(1), X(2), X(3), FVAL);
title('Convergence Plot');
xlabel('Iteration');
ylabel('First-Order Optimality');
set(gca, 'FontSize', 18);
set(gcf, 'Position', [100, 100, 1600, 900]);
exportgraphics(gcf, 'Problem1_tr.pdf', 'ContentType', 'vector');

% Pattern Search Algorithm

f = @(x) 10*x(1)^3 + x(1)*x(2)^2 + x(3)*(x(1)^2 + x(2)^2);
nonlcon = @(x) deal([ sqrt(x(1)^2 + x(2)^2) - x(3) - 10;
                      sqrt(x(1)^2 + x(2)^2) + x(3) - 3 ],  []);
options = optimoptions('patternsearch', ...
    'Display', 'iter', ...
    'MaxIterations', 500);
x0 = [0, -5, 0];

[X, FVAL] = patternsearch(f, x0, [], [], [], [], [], [], nonlcon, ...
    options);

fprintf('Optimal Solution: x1 = %.4f, x2 = %.4f, x3 = %.4f, f = ...
    %.6f\n', X(1), X(2), X(3), FVAL);

% Simulated Annealing Algorithm with Penalty Function Method

clc; clear; close all;

f_original = @(x) 10*x(1)^3 + x(1)*x(2)^2 + x(3)*(x(1)^2 + x(2)^2);
nonlcon = @(x) [ sqrt(x(1)^2 + x(2)^2) - x(3) - 10;
                 sqrt(x(1)^2 + x(2)^2) + x(3) - 3 ];
P = 1e6;
f = @(x) f_original(x) + P * sum(max(nonlcon(x), 0).^2);
lb = [-20, -20, -20];
ub = [20, 20, 20];
options = optimoptions('simulannealbnd', ...
```

```
    'MaxIterations', 280, ...
    'FunctionTolerance', 1e-12, ...
    'PlotFcn', @saplotbestf);

[X, FVAL] = simulannealbnd(f, [0, -5, 0], lb, ub, options);

fprintf('Optimal Solution: x1 = %.4f, x2 = %.4f, x3 = %.4f, f = ...
    %.6f\n', X(1), X(2), X(3), FVAL);
title('Simulated Annealing');
xlabel('Iteration');
ylabel('Fval');
set(gca, 'FontSize', 18);
set(gcf, 'Position', [100, 100, 1600, 900]);
exportgraphics(gcf, 'Problem1_sa.pdf', 'ContentType', 'vector');

diary off;
```

Problem 3

x (AU)	y (AU)
5.764	0.648
6.286	1.202
6.759	1.823
7.168	2.526
7.480	3.360

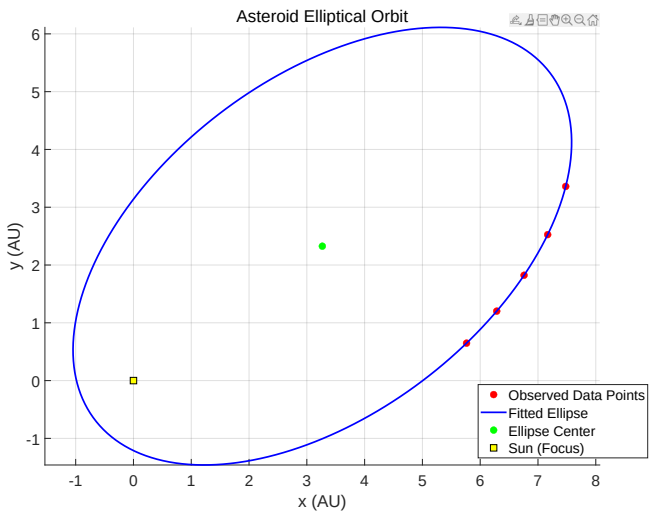

Figure 4: Problem 3

Elliptical Orbit: $0.069373x^2 - 0.075063xy + 0.090058y^2 - 0.278865x - 0.173594y - 0.342434 = 0$.
Ellipse Center: $(x_0, y_0) = (3.268191, 2.325810)$.
Semi-Major Axis: $a = 4.951658$, Semi-Minor Axis: $b = 2.903180$.

```
% ----------Problem 3----------
diary('output3.txt');
diary on;

clc; clear; close all;

% Observed data points (x, y) in AU
x = [5.764, 6.286, 6.759, 7.168, 7.480]';
y = [0.648, 1.202, 1.823, 2.526, 3.360]';

% Initial guess: [a, b, x0, y0, theta] - semi-major axis, ...
    semi-minor axis, ellipse center, and rotation angle (radians)
params0 = [4, 2, 6.5, 2, 0];
options = optimoptions('lsqnonlin', 'Display', 'iter', ...
    'TolFun', 1e-12, 'TolX', 1e-12);

% Define residual function with constraint: one focus at (0,0)
residual_fun = @(p) ellipse_residual_with_focus(p, x, y);

% Solve using nonlinear least squares
[params_fit, resnorm] = lsqnonlin(residual_fun, params0, [], [], ...
    options);

% Extract fitted parameters
a = params_fit(1);      % Semi-major axis
b = params_fit(2);      % Semi-minor axis
x0 = params_fit(3);     % Ellipse center x
y0 = params_fit(4);     % Ellipse center y
theta = params_fit(5); % Rotation angle in radians
c = sqrt(a^2 - b^2);    % Distance from center to focus

% Plot the Orbit
theta_grid = linspace(0, 2*pi, 200); % Angle parameterization
X_ellipse = a * cos(theta_grid);      % Parametric x (unrotated)
Y_ellipse = b * sin(theta_grid);      % Parametric y (unrotated)

% Apply rotation matrix
R = [cos(theta), -sin(theta); sin(theta), cos(theta)];
ellipse_rotated = R * [X_ellipse; Y_ellipse];

% Translate ellipse to its center
X_ellipse = ellipse_rotated(1, :) + x0;
Y_ellipse = ellipse_rotated(2, :) + y0;

% Plot original data points, fitted ellipse, center, and focus
figure; hold on; axis equal;
plot(x, y, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 8);        ...
    % Data points
plot(X_ellipse, Y_ellipse, 'b-', 'LineWidth', 2);                 ...
    % Fitted ellipse
```

```
plot(x0, y0, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 8);      % Ellipse center
plot(0, 0, 'ks', 'MarkerFaceColor', 'y', 'MarkerSize', 10);       % Sun (fixed focus at origin)

legend('Observed Data Points', 'Fitted Ellipse', ...
       'Ellipse Center', 'Sun (Focus)', 'Location', 'Best');
xlabel('x (AU)');
ylabel('y (AU)');
title('Asteroid Elliptical Orbit');
grid on;

% Save and Print
set(gca, 'FontSize', 18);
set(gcf, 'Position', [100, 100, 1600, 900]);
exportgraphics(gcf, 'asteroid_orbit.pdf', 'ContentType', 'vector');

fprintf('Fitted Ellipse Parameters:\n');
fprintf('  Semi-Major Axis a = %.6f\n', a);
fprintf('  Semi-Minor Axis b = %.6f\n', b);
fprintf('  Ellipse Center: (%.6f, %.6f)\n', x0, y0);
fprintf('  Rotation Angle (rad): %.6f\n', theta);
fprintf('  Distance from Center to Focus: %.6f\n', c);
fprintf('  Actual Distance to Origin: %.6f\n', sqrt(x0^2 + y0^2));

diary off;

function r = ellipse_residual_with_focus(p, x, y)
    % Extract ellipse parameters
    a = p(1); b = p(2);
    x0 = p(3); y0 = p(4);
    theta = p(5);

    % Compute geometric constraint: focus should be at origin
    c = sqrt(a^2 - b^2);                    % Distance from center to focus
    focus_error = sqrt(x0^2 + y0^2) - c;  % Difference from origin
    focus_penalty = 10 * focus_error;     % Enforce this as soft constraint (scaled)

    % Rotate and translate points into ellipse's coordinate system
    cos_t = cos(theta); sin_t = sin(theta);
    x_shifted = x - x0;
    y_shifted = y - y0;
    x_rot = cos_t * x_shifted + sin_t * y_shifted;
    y_rot = -sin_t * x_shifted + cos_t * y_shifted;

    % Algebraic ellipse equation residuals: should be close to 1
    ellipse_eq = (x_rot / a).^2 + (y_rot / b).^2 - 1;

    % Combine residuals
    r = [ellipse_eq; focus_penalty];
end

% Output:
% Fitted Ellipse Parameters:
%   Semi-Major Axis a = 4.951658
```

```
% Semi-Minor Axis b = 2.903180
% Ellipse Center: (3.268191, 2.325810)
% Rotation Angle (rad): 0.650951
% Distance from Center to Focus: 4.011292
% Actual Distance to Origin: 4.011292
```

Problem 4

$$\min f(x) = -c_1 e^{-0.2\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}} - e^{\frac{1}{n}\sum_{i=1}^{n}\cos(2\pi x_i)} + c_1 + e, \quad c_1 = 20, \quad n = 1, 2, 10.$$

For all n, $f(x)_{\min} = 0$ and $x = 0$. As for $n = 10$, prior trials have shown that the optimal results often converge to a local optimum. Therefore, we choose to dynamically adjust the cognitive parameter and social parameter, and increase the number of particles and iterations (even though the convergence is good). We increase `max_iter` to 500 and `n_particles` to 10000. In early stage, we increase `c1` to let particles explore more possible solutions. In late stage, we enlarge `c2` to let particles focus on the optimal solution, improving the convergence speed. For the video demonstration at $n = 1$, click here.

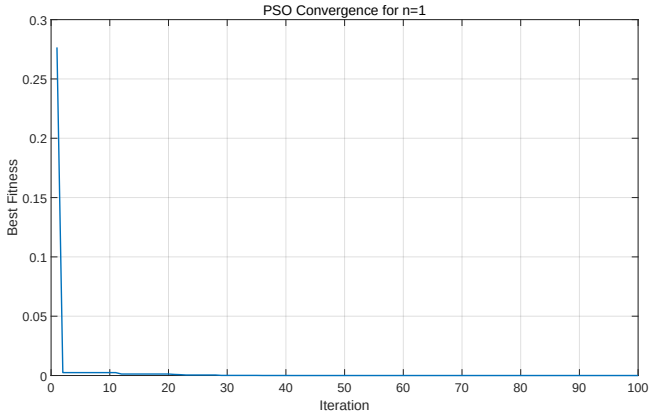

Figure 5: Problem 4, $n = 1$

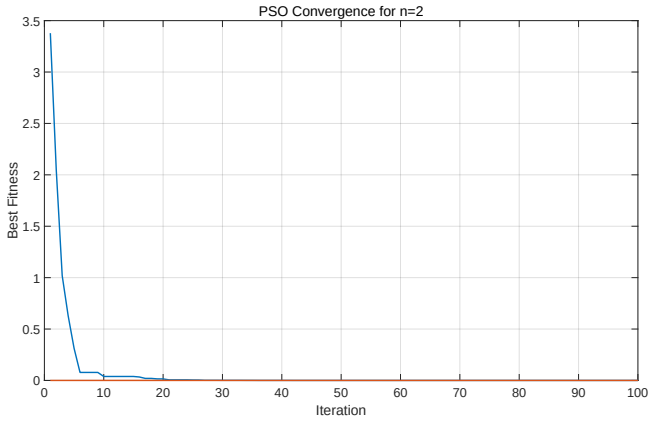

Figure 6: Problem 4, $n = 2$

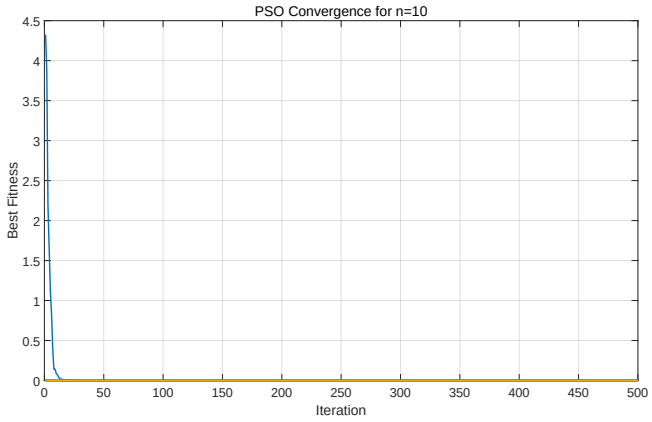

Figure 7: Problem 4, $n = 10$

```
%----------Problem 4----------
warning('off', 'all');
diary('output4.txt');
diary on;

clc; clear; close all;

n = [1, 2, 10];
n_particles = 100;
max_iter = 100;
w = 0.7;              % Inertia Weight
c1 = 1.5;             % Cognitive Parameter
c2 = 1.5;             % Social Parameter
bounds = [-5, 5];     % Search Range
```

```
15
16 % Ackley function
17 ackley = @(x) -20 * exp(-0.2 * sqrt(sum(x.^2) / length(x))) ...
18               - exp(sum(cos(2*pi * x)) / length(x)) + 20 + exp(1);
19
20 for i = n
21     fprintf('Running PSO for n = %d\n', i);
22
23     if i == 10
24         % More Particles and Iterations
25         n_particles = 10000;
26         max_iter = 500;
27     end
28
29     % Initialize particles
30     positions = bounds(1) + (bounds(2) - bounds(1)) * ...
           rand(n_particles, i);
31     velocities = zeros(n_particles, i);
32     pbest = positions;
33     gbest = positions(1, :);
34     pbest_fitness = zeros(n_particles, 1);
35     for j = 1:n_particles
36         pbest_fitness(j) = ackley(pbest(j, :));
37     end
38     gbest_fitness = ackley(gbest);
39     history = zeros(max_iter, i);
40
41     % Initialize a Video Object
42     if i == 1
43         video_filename = 'pso_optimization';
44         v = VideoWriter(video_filename, 'MPEG-4');
45         v.FrameRate = 10;
46         open(v);
47     end
48
49     for iter = 1:max_iter
50         if i == 10
51             % Early Stage (large c1): Particles Explore More ...
                   Possible Solutions.
52             c1 = 2.5 - iter * (2.5 - 0.5) / max_iter;
53             % Late Stage (large c2): Particles Focus on the ...
                   Optimal Solution, Improving the Convergence Speed.
54             c2 = 0.5 + iter * (2.5 - 0.5) / max_iter;
55             w = 0.4;
56         end
57
58         for j = 1:n_particles
59             p = positions(j, :);
60             fitness = ackley(p);
61
62             % Update Personal Best
63             if fitness < pbest_fitness(j)
64                 pbest(j, :) = p;
65                 pbest_fitness(j) = fitness;
66             end
67
68             % Update Global Best
```

```
            if fitness < gbest_fitness
                gbest = p;
                gbest_fitness = fitness;
            end
        end

        for j = 1:n_particles
            r1 = rand();
            r2 = rand();

            % Update Velocity
            velocities(j, :) = w * velocities(j, :) + ...
                               c1 * r1 * (pbest(j, :) - ...
                                   positions(j, :)) + ...
                               c2 * r2 * (gbest - positions(j, :));

            % Update Position
            new_position = positions(j, :) + velocities(j, :);

            % Boundary handling
            new_position = max(min(new_position, bounds(2)), ...
                bounds(1));

            positions(j, :) = new_position;
        end

        history(iter) = gbest_fitness;

        % Visualization
        if i == 1
            clf;
            fplot(ackley, bounds, 'k-', 'LineWidth', 2);
            hold on;
            scatter(positions, arrayfun(ackley, positions), 'ro', ...
                'filled');
            scatter(gbest, ackley(gbest), 'b*', 'LineWidth', 2);
            title(sprintf('Iteration %d, Best Value: %.4f', iter, ...
                gbest_fitness));
            xlabel('x');
            ylabel('f(x)');
            frame = getframe(gcf);
            writeVideo(v, frame);
            pause(0.05);
        end
    end

    if i == 1
        close(v);
        fprintf('Video saved as %s\n', video_filename);
    end

    % Print the Final Result
    fprintf('Best solution found for n=%d: %.6f\n', i, ...
        gbest_fitness);

    % Plot Convergence Curve
    figure;
```

```
121         plot(history, 'LineWidth', 2);
122         xlabel('Iteration');
123         ylabel('Best Fitness');
124         title(sprintf('PSO Convergence for n=%d', i));
125         grid on;
126 
127         set(gca, 'FontSize', 18);
128         set(gcf, 'Position', [100, 100, 1600, 900]);
129         exportgraphics(gcf, sprintf('Problem4_n%d.pdf', i), ...
                'ContentType', 'vector');
130     end
131 
132     diary off;
133 
134     % Output:
135     % Running PSO for n = 1
136     % Video saved as pso_optimization.avi
137     % Best solution found for n=1: 0.000000
138     % Running PSO for n = 2
139     % Best solution found for n=2: 0.000000
140     % Running PSO for n = 10
141     % Best solution found for n=10: 0.000000
```